

מטלה שבועית – מנוע פיסיקלי וארכיטקטורת קוד

במטלה זו עליכם לתכנת משחקון הכולל שימוש בפיסיקה. מטרת המטלה היא לאמן אתכם בשני נושאים:

- המנוע הפיסיקלי של יוניטי. ניתן להיעזר בקוד שהוצג בהרצאה, שנמצא בגיטהאב כאן: <https://github.com/gamedev-at-ariel>
- הנדסת תוכנה: תיכנון נכון של מחלקות.

הגשה

- העלו את המשחק לחשבון איץ' של הצוות. ודאו שאפשר להריץ אותו גם מדפדפן אחר / מחשב אחר. הוסיפו "תיאור" למשחק ובו הסבר על אופן המשחק וקישור לגיטהאב.
- העלו את המשחק לגיטהאב של הצוות. הוסיפו רידמי מפורט המסביר מה עשיתם ואיך בדיוק פתרתם את הבעיות - באיזה אלגוריתמים, רכיבים, טכניקות וכו' השתמשתם. הוסיפו קישורים לשורות-הקוד הרלבנטיות, וקישור למשחק באיץ'.
- הוסיפו את הקובץ `linter.yml` שנמצא בגיטהאב של הקורס, לצורך בדיקת איכות הקוד.

רשימת משחקים לבחירה

יש לבחור אחד מהבאים:

משחק 1: [The Incredible Machine](#)

משחק של חידות פיסיקליות. השחקן צריך לבנות מכונה פיסיקלית מרכיבים נתונים, ע"י גרירת רכיבים למשטח. המכונה צריכה לבצע פעולה מוגדרת מראש. לדוגמה: צריך לזרוק כדור שמפיל דומינו והדומינו מפיל עוד דברים וכו'.

משחק 2: [SleepWalker](#)

משחק שבו ילד הולך מתוך שינה, והכלב החכם שלו צריך להציל אותו מנפילות ולהחזיר אותו הביתה בשלום. הוא מוצא רכיבים פיסיקליים ויכול להשתמש בהם, כגון חבלים, קפיצים וכו'.

משחק 3: הנחתת חללית

בנו משחק שבו מטרת השחקן היא להנחית חללית בבטחה על-גבי כוכב-לכת, בלי להתנגש בסלעים ובמכשולים. השחקן לא שולט במיקום החללית באופן ישיר, אלא השליטה היא עקיפה ע"י הוספת כוחות:

- חץ למעלה – מוסיף לחללית כוח בכיוון החרטום שלה.
- חץ למטה – מוסיף לחללית כוח בכיוון הזנב שלה.
- חץ ימינה – מוסיף לחללית כוח סיבובי (Torque) ימינה;
- חץ שמאלה – מוסיף לחללית כוח סיבובי שמאלה.

יש מכשולים באויר ועל הקרקע, שצריך להתחמק מהם.

משחק 4: [מארינו](#)

משחק פלטפורמה, דומה למשחק שבנינו בשיעור (הסצינות בתיקיות 1 ו-2) אבל עם עוד כמה תכונות פיסיקליות, כגון: אם השחקן נופל ופוגע ברצפה במהירות גבוהה מדי – הוא שובר רגל, וכוח-ההליכה שלו קטן פי 2 למשך מספר שניות; מארינו יכול למצוא נעליים קפיציות מיוחדות. כשהוא נועל אותן, הוא יכול לקפוץ מגובה בלי לשבור רגל, והנעליים יקפיצו אותו לגובה יותר; באיזור מסויים במשחק יש רוח חזקה, המפעילה כוח על השחקן ומאטה את ההתקדמות שלו; ליד אחד המשטחים הגבוהים יש עמוד. במקום לקפוץ מהמשטח ולשבור רגל, מארינו יכול להחליק על העמוד ולהגיע לקרקע בבטחה (כמו כאן <https://youtu.be/rLl9XBg7wSs?t=130>).

משחק 5 : באולינג

משחק דמוי-באולינג לשחקן אחד :

- השחקן מגלגל כדור כבד לכיוון הבקבוקים. המשחק מחשב כמה בקבוקים נפלו ומסלק אותם מהזירה.
 - השחקן מגלגל שוב, נפלו המשחק מחשב כמה בקבוקים מהנשארים נפלו.
 - אם כל הבקבוקים אחרי שני גלגולים – השחקן עובר לרמה הבאה ; אחרת הוא חוזר לתחילת הרמה הנוכחית.
- מספר הבקבוקים והסידור שלהם לא חייב להיות כמו במשחק הרשמי.

משחק 6 : Angry Birds

השחקן יורה ציפור מרוגטקה (הוא יכול לשלוט בכוח שהוא מפעיל ובכיוון היריה), ומטרתו שהציפור "תחסל" את החזירים ע"י פגיעה בהם. השחקן עובר לרמה הבאה לאחר שהוא מחסל את כל החזירים ברמה הנוכחית. החזירים יושבים בתוך מבנה מורכב, כדי לחסל אותם צריך להפיל את המבנה. אפשר להעזר בסצנה AngryBall.unity שנמצאת בגיטהאב בתיקיה 5 (basketball).

משחק 7: ביליארד

משחק דמוי ביליארד, עם חוקים דומים למשחק הרשמי או חוקים חדשים לבחירתכם.

משחק 8: ?

ניתן לבחור משחק מקורי אחר כלשהו שיש בו תוכן פיסיקלי משמעותי (אפקטורים, מחברים וכו').

ארכיטקטורת קוד

במטלה זו יש להקפיד על עיצוב מחלקות ועקרונות **OOP**. דוגמה למחלקות (עבור משחק דמוי מריו):

- **Player Class**: אחראית על תנועה, גילוי התנגשויות (Collision) עם חפצים ואויבים, וניהול החיים.

- **Enemy Class**: אחראית על תנועת אויב (ניתן ליצור EnemyBase ולהוריש תנועות שונות במחלקות בנות)

- **Collectible Class**: אחראית על טיפול בסוג החפץ, אפשרויות Spawn, ואינטראקציה עם השחקן.

- **UI Manager**: אחראי על עדכוני הממשק הגרפי (ציון, חיים, מסכי סיום).

2. ארגון הסקריפטים בתיקיות. דוגמה למבנה אפשרי:

```
Assets/  
├── Scripts/  
│   ├── Managers/  
│   │   ├── GameManager.cs  
│   │   └── SpawnManager.cs  
│   ├── Player/  
│   │   └── PlayerController.cs  
│   ├── Enemies/  
│   │   └── EnemyBase.cs  
│   ├── Collectibles/  
│   │   └── CollectibleItem.cs  
│   ├── UI/  
│   │   └── UIManager.cs  
│   └── Utils/  
│       ├── Timer.cs  
│       └── Constants.cs  
├── Prefabs/  
├── Scenes/  
└── UI/
```

3. שמירה על קוד נקי (Clean Code)

- כל מחלקה צריכה להיות בעלת אחריות ממוקדת (Single Responsibility).
- שמות משתנים ופונקציות צריכים להיות תיאוריים וברורים.
- הימנעו ממחלקות ענקיות
- מומלץ לכתוב הערות קצרות (Comments) רק היכן שהדבר תורם להבנה, ולא באופן מוגזם.

תהליך עבודה (Workflow)

1. תכנון מבנה המחלקות

- צרו תרשים UML בסיסי או שרטוט על דף שמראה איך המחלקות מתקשרות זו עם זו.
- החליטו איך המשחק יידע לעדכן את UIManager כאשר השחקן אוסף חפץ, וכו'.
- 2. מימוש מכניקות בסיסיות, לדוגמה (עבור משחק מריו):
 - צרו PlayerController שמאפשר תנועה ובודק התנגשויות פשוטות עם הסביבה.
 - צרו מחלקה בסיסית לאויב (EnemyBase) עם לוגיקת תנועה מינימלית.
 - צרו CollectibleItem שניתן לאסוף ושהוא מעלה ניקוד.

3. הוספת "פוליש" (Refinements)

- ודאו שהתנאי לניצחון/הפסד פועל: אם הזמן נגמר, או אם אספתם מספיק חפצים וכו'.
- הוסיפו מסך סיום (Game Over) או מסך ניצחון (Win Screen).

4. Refactor - שיפור מבנה הקוד

- אם יש קוד שחוזר על עצמו (כמו אלגוריתם תנועה לאויבים שונים), שקלו להשתמש במחלקת אב משותפת או ממשק (Interface).
- הקפידו שכל סקריפט נשאר ממוקד יחסית ואינו מתנפח ליותר מדי פונקציות ותכונות.

הגשה

1. ריפודיטורי מסודר ב **GitHub**
2. **README** קצר שמסביר:
 - איך מריצים את המשחק.
 - תיאור קצר של יחסי המחלקות (Class Relationships).
 - הנחות שבוצעו או בחירות מיוחדות בארכיטקטורה.
3. תרשים מחלקות (UML) או תרשים בסיסי אחר, שמציג את המחלקות והקשרים החשובים ביניהן.
4. בניה והעלאת המשחק ל-Itch.io בפורמט WebGL עם פירוט הוראות איך לשחק ולינקים לריפודיטורי מ-Itch ומה-Itch לריפודיטורי.